

CS223 UART DESIGN PROJECT

Name: Barış Pamuksuz

ID: 22202238

Section: 1

TASK 1: DESIGN AND FUNCTIONALITY

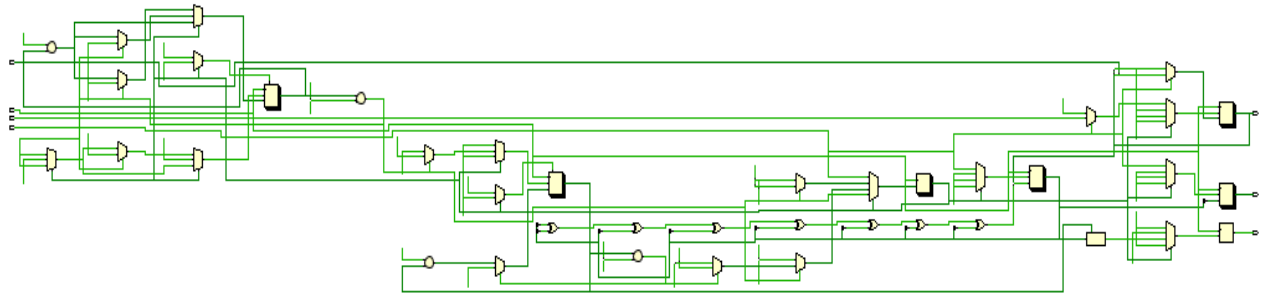
Main UART (part 1) requires basically TX and RX buffers implementation. To establish same bit sending-receiving, same period should be used. Period is determined by the baud rate, that I took 115200 in my code. Hence $\text{CLKS_PER_BIT} = 100\text{MHz}/115200 = 868$, meaning that at the end of 868 cycle, new bit is sent or obtained. Both of buffers have 4 states. They are idle, start, get/send bit and stop states. For TX buffer, when state is Idle, logic is 1. Whenever user press BTNC, TX goes to start state and sends 0 signal to RX buffer. Since RX buffer too was in idle state, incoming 0 signal initiates state cycling by moving start state to be able to receive a byte. RX and TX buffers move into get bit and send bit states respectively. After sending data, RX checks the parity bit generated by TX according to number of 1s. In my case, I used even parity. If income data holds even number of 1s and 0 as parity or odd number of 1s and 1 as parity, data receive is completed and transmitted to stop state that is in logic high signal. After a while, again, returned to idle state for both buffers.

In second part, FIFO registers are completed by defining 32-bit memories for both buffers. For TX, when loadButton is pressed, new data is loaded to first 8 bits, that is TX[0], and TX[3] is discarded with the convention that rest of data are sided by one to the right. Discarded data is sent through state mechanism to RX. For RX, after get bit state, that is completing a 8 bit sequence successfully, obtained data is set to RX[0] and R[3] is discarded with the convention of moving income data by one to the right. Since button inputs might cause noise or unwanted input streams, I implemented button debounce to ensure pushing on button generates 1 signal not constantly.

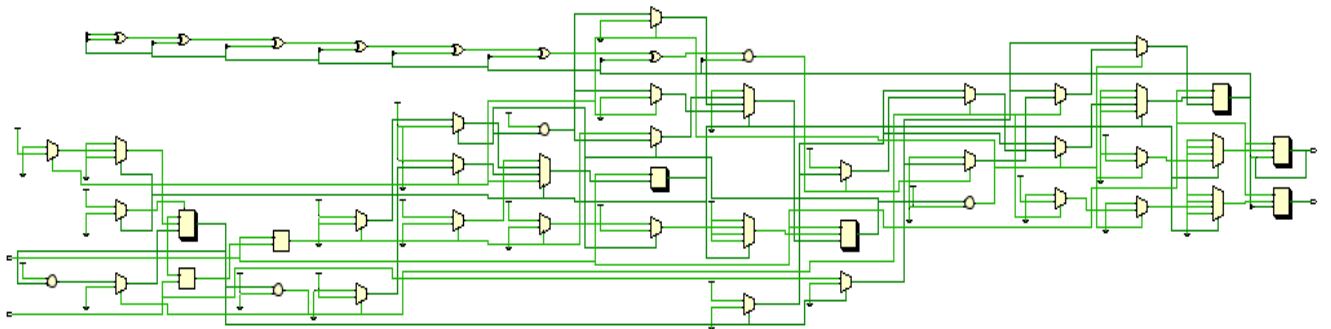
In third part, to get hexadecimal values of data inside memories, I divided each byte into 2 nibbles. Each nibble corresponding to later displayed hexadecimal symbols. Other 2 digits are assigned to determine in which memory location 7-segment-display shows and the index of data inside those memories.

TASK 2: RTL SCHEMATICS

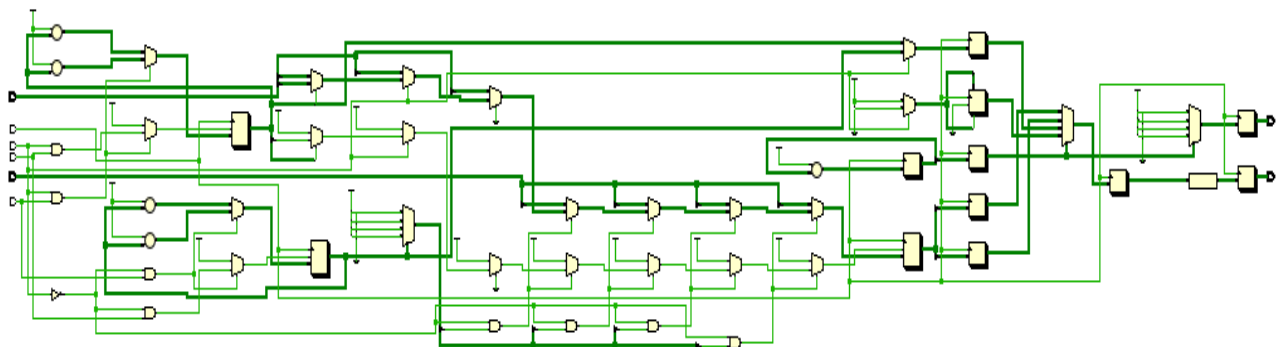
TX RTL



RX RTL

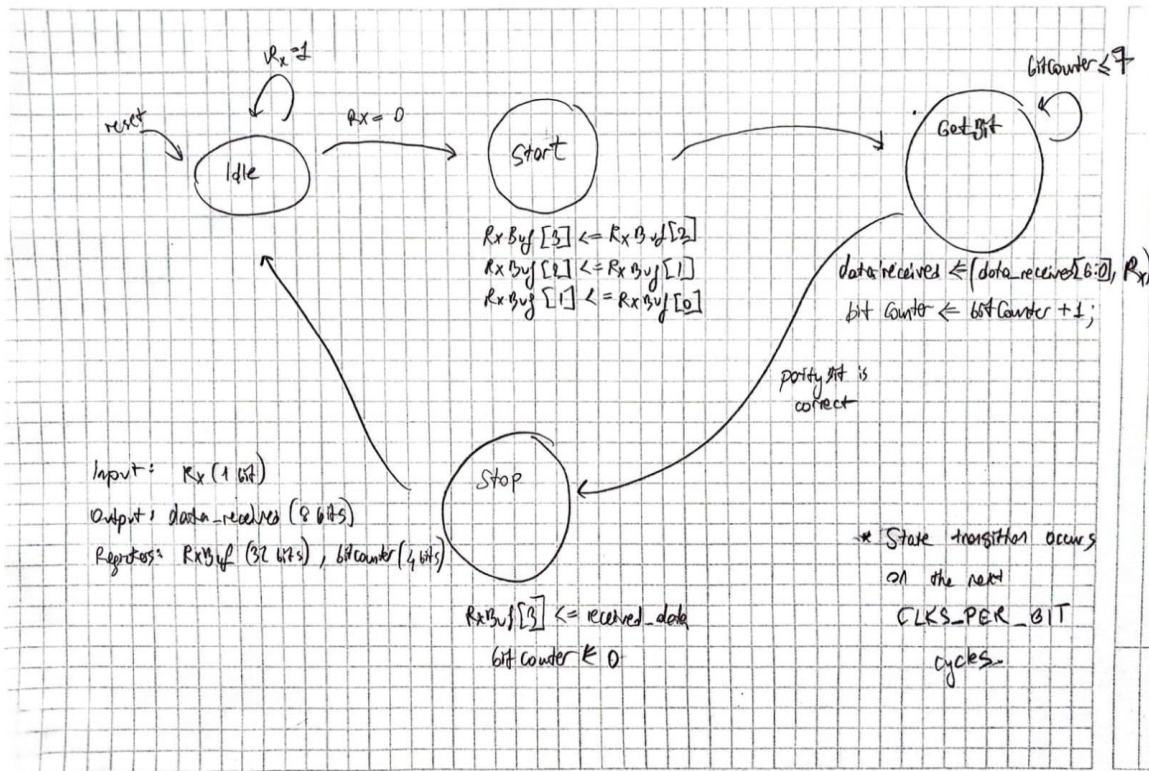


7SegmentDisplay RTL

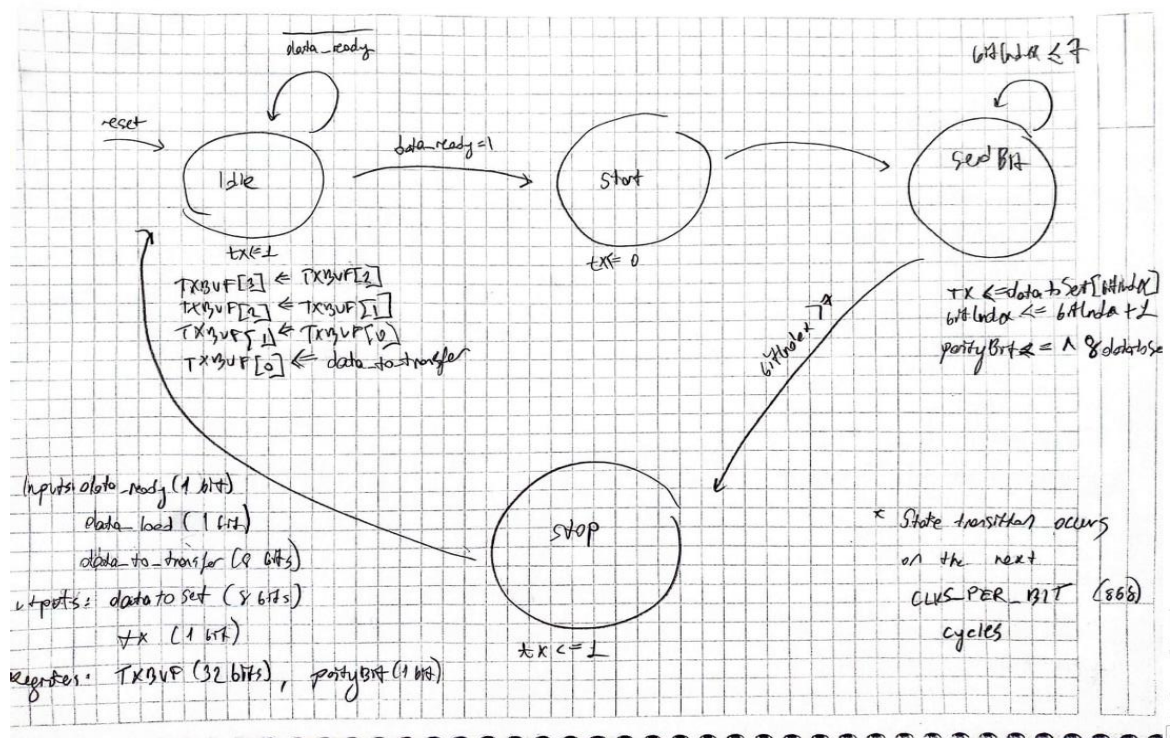


TASK 3: STATE DIAGRAMS

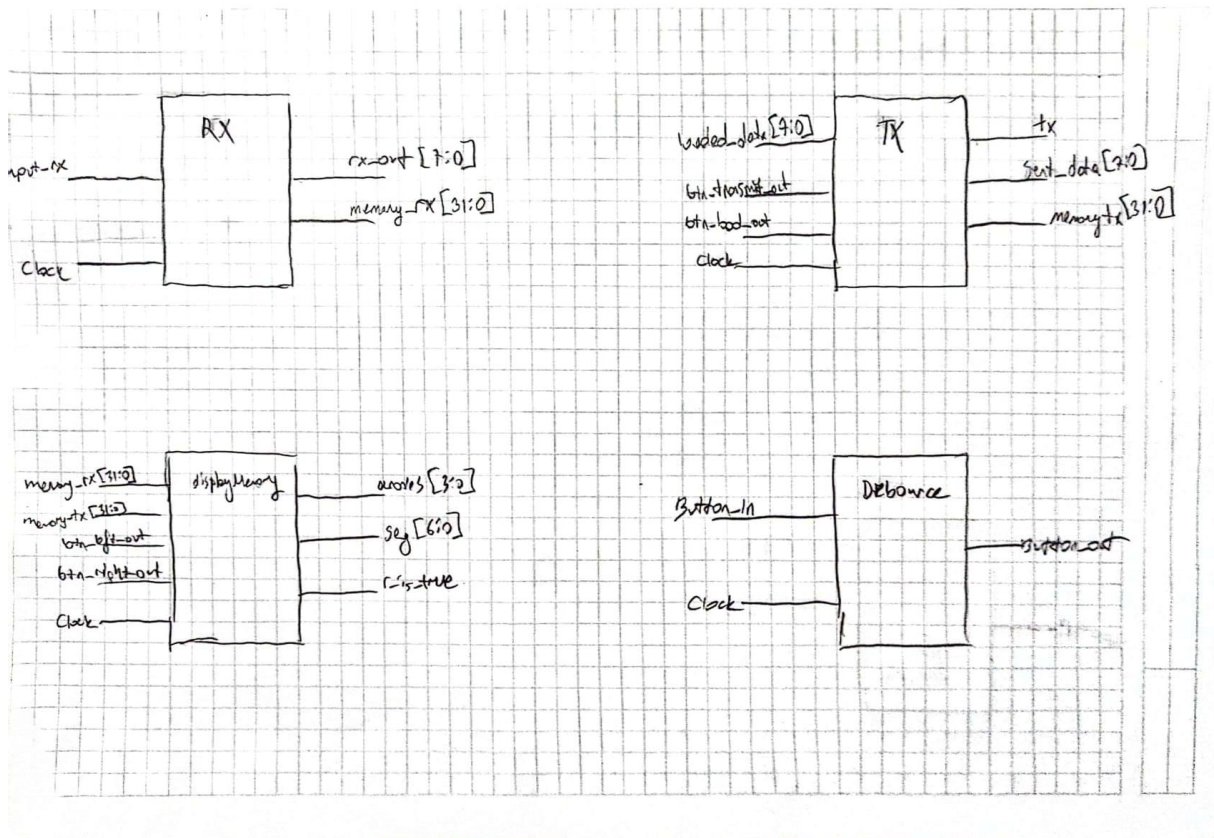
RX



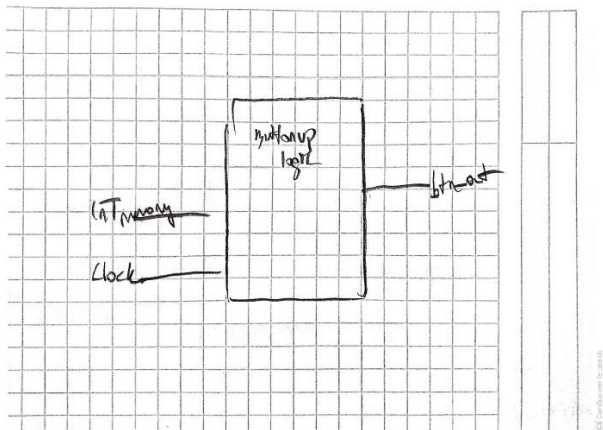
TX



TASK 4: BLOCK DIAGRAM FOR EACH MODULES



CS CamScanner ile tarandı



TASK 5: TESTBENCH RX – TX

TESTBENCH TX MODULE

``timescale 1ns / 1ps`

`module testbench;`

```
reg [7:0] data_to_Transfer;
reg data_ready;
reg loadButton;
reg clock;
wire tx;
wire[7:0] mostRecent; // Assuming this is also an output now
```

```
Uart_tx uart_tx_inst (
    .data_to_Transfer(data_to_Transfer),
    .data_ready(data_ready),
    .loadButton(loadButton),
    .clock(clock),
    .tx(tx),
    .mostRecent(mostRecent)    // Assuming these are correctly declared as outputs in the
Uart_tx
);
```

```
// Clock generation
```

```
initial begin
```

```
    clock = 0;
```

```
    forever #5 clock = ~clock;
```

```
end
```

```
// Test sequence
```

```
initial begin
```

```
    // Initialize inputs
```

```
    data_to_Transfer = 0;
```

```
    data_ready = 0;
```

```
loadButton = 0;
```

```
#100;
```

```
// Test 1: Load data into FIFO
```

```
@(posedge clock);
```

```
data_to_Transfer = 8'hAA; // Example data
```

```
loadButton = 1;
```

```
@(posedge clock);
```

```
loadButton = 0;
```

```
// Test 2: Load another data
```

```
@(posedge clock);
```

```
data_to_Transfer = 8'hBB; // Example data
```

```
loadButton = 1;
```

```
@(posedge clock);
```

```
loadButton = 0;
```

```
// Test 3: Load additional data while sending
```

```
repeat(10) @(posedge clock);
```

```
data_to_Transfer = 8'hDD;
```

```
loadButton = 1;
```

```
@(posedge clock);
```

```
loadButton = 0;
```

```
// Test 4
```

```
repeat(5) @(posedge clock);
```

```
data_to_Transfer = 8'hEE;
```

```

loadButton = 1;
@(posedge clock);
data_to_Transfer = 8'hFF;
@(posedge clock);
loadButton = 0;

repeat(10) @(posedge clock);
data_ready = 1;
@(posedge clock);
data_ready = 0;

#100
@(posedge clock);
loadButton = 1;
@(posedge clock);
loadButton = 0;
// Continue simulation to observe full behavior
#200;

// End simulation
$finish;
end
endmodule

TESTBENCH RX MODULE
`timescale 1ns / 1ps

module testbench2;

reg i_rx;

```

```

reg clock;
reg [7:0] memoryR;

Uart_rx uart_rx_inst (
    .i_rx(i_rx),
    .clock(clock),
    .data_receivedOldest(memoryR)
);

// Clock generation
initial begin
    clock = 0;
    forever #5 clock = ~clock; // 100 MHz clock
end

// Simulate incoming data bits
initial begin
    memoryR = 8'b0;
    // Initialize
    i_rx = 1;
    #100;

    // START_STATE bit
    i_rx = 0;
    #86.8;

    // Data bits for 0x55 -> 01010101
    i_rx = 1; #86.8;
    i_rx = 0; #86.8;

```

```
i_rx = 1; #86.8;
i_rx = 0; #86.8;
i_rx = 1; #86.8;
i_rx = 0; #86.8;
i_rx = 1; #86.8;
i_rx = 0; #86.8;
i_rx = 1; #86.8; // Parity (even)
i_rx = 1; #86.8; // STOP_STATE bit
i_rx = 1; #173.6;
```

```
// Send 0xAA -> 10101010
```

```
i_rx = 0; #86.8;
i_rx = 0; #86.8; // Bit 0
i_rx = 1; #86.8; // Bit 1
i_rx = 0; #86.8; // Bit 2
i_rx = 1; #86.8; // Bit 3
i_rx = 0; #86.8; // Bit 4
i_rx = 1; #86.8; // Bit 5
i_rx = 0; #86.8; // Bit 6
i_rx = 1; #86.8; // Bit 7
i_rx = 0; #86.8; // Parity bit (odd)
i_rx = 1; #86.8; // STOP_STATE bit
i_rx = 1; #173.6; // IDLE_STATE state
```

```
// Send 0xFF -> 11111111 to test all high bits
```

```
i_rx = 0; #86.8; // START_STATE bit
i_rx = 1; #86.8; // Bit 0
i_rx = 1; #86.8; // Bit 1
i_rx = 1; #86.8; // Bit 2
```

```
i_rx = 1; #86.8; // Bit 3  
i_rx = 1; #86.8; // Bit 4  
i_rx = 1; #86.8; // Bit 5  
i_rx = 1; #86.8; // Bit 6  
i_rx = 1; #86.8; // Bit 7  
i_rx = 0; #86.8; // Parity bit (even)  
i_rx = 1; #86.8; // STOP_STATE bit  
i_rx = 1; #173.6; // IDLE_STATE state
```

```
i_rx = 0; #86.8; // START_STATE bit  
i_rx = 1; #86.8; // Bit 0  
i_rx = 0; #86.8; // Bit 1  
i_rx = 1; #86.8; // Bit 2  
i_rx = 0; #86.8; // Bit 3  
i_rx = 1; #86.8; // Bit 4  
i_rx = 0; #86.8; // Bit 5  
i_rx = 1; #86.8; // Bit 6  
i_rx = 0; #86.8; // Bit 7  
i_rx = 1; #86.8; // Parity bit (even)  
i_rx = 1; #86.8; // STOP_STATE bit  
i_rx = 1; #173.6; // IDLE_STATE state
```

```
i_rx = 0; #86.8; // START_STATE bit  
i_rx = 1; #86.8; // Bit 0  
i_rx = 0; #86.8; // Bit 1  
i_rx = 1; #86.8; // Bit 2  
i_rx = 0; #86.8; // Bit 3  
i_rx = 1; #86.8; // Bit 4  
i_rx = 0; #86.8; // Bit 5
```

```

i_rx = 1; #86.8; // Bit 6
i_rx = 0; #86.8; // Bit 7
i_rx = 1; #86.8; // Parity bit (even)
i_rx = 1; #86.8; // STOP_STATE bit
i_rx = 1; #173.6; // IDLE_STATE state

$finish;

end

```

```
endmodule
```

APPENDIX

TX

```

module tx #(parameter CLKS_PER_BIT = 868)(
input logic clock,
input logic btn_load_out,
input logic transmit_button,
input logic [7:0] loaded_data,
output logic tx,
output logic [7:0] sent_data,
output logic [31:0] txMemory
);

logic parity_bit;

reg [15:0] counter = 0;
reg [8:0] sent2_data = 0;

localparam idle = 2'b00;
localparam start = 2'b01;
localparam send_bit = 2'b10;

```

```
localparam stop = 2'b11;
```

```
reg [1:0] state = 0;
```

```
reg [3:0] bit_index = 0;
```

```
always @(posedge clock) begin
```

```
    case(state)
```

```
        idle: begin
```

```
            tx <= 1;
```

```
            counter <= 0;
```

```
            bit_index <= 0;
```

```
        if(btn_load_out)begin
```

```
            txMemory[31:24] <= txMemory[23:16];
```

```
            txMemory[23:16] <= txMemory[15:8];
```

```
            txMemory[15:8] <= txMemory[7:0];
```

```
            txMemory[7:0] <= loaded_data[7:0];
```

```
        end
```

```
        if(transmit_button) begin
```

```
            sent2_data[7:0] <= txMemory[31:24];
```

```
            sent_data[7:0] <= sent2_data[7:0];
```

```
            parity_bit = sent2_data[0]^sent2_data[1]^sent2_data[2]^sent2_data[3]^
```

```
            sent2_data[4]^sent2_data[5]^sent2_data[6]^sent2_data[7];
```

```
            sent2_data[8] <= parity_bit;
```

```
            txMemory[31:24] <= txMemory[23:16];
```

```
            txMemory[23:16] <= txMemory[15:8];
```

```
            txMemory[15:8] <= txMemory[7:0];
```

```
            txMemory[7:0] <= 0;
```

```

        state <= start;
    end
    else begin
        state <= idle;
    end
end

start: begin
    tx <= 0;
    if(counter < (CLKS_PER_BIT - 1)) begin
        counter = counter + 1;
        state <= start;
    end
    else begin
        counter <= 0;
        state <= send_bit;
    end
end

send_bit: begin
    tx <= sent2_data[bit_index];
    //led2[7:0] <= sent2_data[7:0];
    if(counter < (CLKS_PER_BIT - 1)) begin
        counter <= counter + 1;
        state <= send_bit;
    end
    else begin
        counter <= 0;
        if(bit_index < 8) begin
            bit_index <= bit_index + 3'b1;
            state <= send_bit;
        end
    end
end

```

```

        end
    else begin
        bit_index <= 0;
        state <= stop;
    end
end
end
stop: begin
    tx <= 1;
    if(counter < (CLKS_PER_BIT - 1)) begin
        counter= counter + 1;
        state <= stop;
    end
    else begin
        state <= idle;
    end
end
default:
    state <= idle;
endcase
end
endmodule

```

RX

```

module rx#(parameter CLKS_PER_BIT = 868)(
    input logic clock,
    input logic input_rx,
    output logic [7:0] rx_out,
    output logic [31:0] memory_rx
);

```

```
logic parityBit;
```

```
localparam IDLE = 2'b00;
```

```
localparam START = 2'b01;
```

```
localparam GET_BIT_STATE = 2'b10;
```

```
localparam STOP_STATE = 2'b11;
```

```
reg rxBuff = 1'b1;
```

```
reg rx = 1'b1;
```

```
reg [1:0] state = 0;
```

```
reg [15:0] counter = 0;
```

```
reg [8:0] data_received = 0;
```

```
reg [3:0] bit_index = 0;
```

```
always @(posedge clock) begin
```

```
    rxBuff <= input_rx;
```

```
    rx <= rxBuff;
```

```
end
```

```
always @(posedge clock) begin
```

```
case(state)
```

```
IDLE: begin
```

```
    counter <= 0;
```

```
    bit_index <= 0;
```

```
    if(rx == 0)
```

```
        state <= START;
```

```
    else
```

```
        state <= IDLE;
```

```
end
```

```

START: begin

    if(counter == (CLKS_PER_BIT-1)/2) begin
        if(rx == 0) begin
            counter <= 0;
            state <= GET_BIT_STATE;
        end
        else begin
            state <= IDLE;
        end
    end
end

else begin
    counter <= counter + 1;
    state <= START;
end

end

GET_BIT_STATE: begin
    data_received[bit_index] <= input_rx;
    if(counter < (CLKS_PER_BIT-1)) begin
        counter <= counter + 1;
        state <= GET_BIT_STATE;
    end
    else begin
        counter <= 0;
        if(bit_index < 8) begin
            bit_index <= bit_index + 3'b1;
            state <= GET_BIT_STATE;
        end
        else begin
            rx_out[7:0] <= data_received[7:0];

```

```

        parityBit =
data_received[0]^data_received[1]^data_received[2]^data_received[3]^
        data_received[4]^data_received[5]^data_received[6]^data_received[7];
        if(parityBit != data_received[8])begin
            data_received <= 0;
        end
        bit_index <= 0;
        state <= STOP_STATE;
    end
end
STOP_STATE: begin
    if(counter < CLKS_PER_BIT-1) begin
        counter <= counter + 16'b1;
        state <= STOP_STATE;
    end
    else begin
        memory_rx[31:24] <= memory_rx[23:16];
        memory_rx[23:16] <= memory_rx[15:8];
        memory_rx[15:8] <= memory_rx[7:0];
        memory_rx[7:0] <= data_received[7:0];
        counter <= 0;
        state <= IDLE;
    end
end
default: state <= IDLE;
endcase
end
endmodule

```

Debounce

```
module debounce(  
input logic clock,  
input logic ButtonIn,  
output logic Button_out  
);  
    logic prevButtonIn = 0;  
    always_ff @(posedge clock)begin  
        prevButtonIn <= ButtonIn;  
        if(ButtonIn && !prevButtonIn)begin  
            Button_out <= 1;  
        end  
        else begin  
            Button_out <= 0;  
        end  
    end  
endmodule
```

ButtonupLogic

```
module ButtonUpLogic(  
input logic clock,  
input logic btn_out,  
output logic InTMemory  
);  
  
    always_ff @(posedge clock)begin  
        if(btn_out)begin  
            InTMemory <= !InTMemory;  
        end  
    end  
end
```

```
endmodule
```

DisplayMemory

```
module displayMemory(  
    input logic clock,  
    input logic rightButtonOut,  
    input logic leftButtonOut,  
    input logic [31:0] txMemory,  
    input logic [31:0] rxMemory,  
    output reg [3:0] anodes,  
    output logic [6:0] segments,  
    input logic r_is_true  
);
```

```
logic [1:0] case_index = 0;  
logic [1:0] counter;  
logic [19:0] refresh = 20'b0;  
logic [1:0] bit_index_tx;  
logic [1:0] bit_index_rx;  
logic [7:0] regDisplay;
```

```
reg [3:0] digit0;  
reg [3:0] digit1;  
reg [3:0] digit2;  
reg [4:0] digit3;  
reg [4:0] led = 0;
```

```
always @(posedge clock)
```

```

begin
    digit0 <= regDisplay[3:0];
    digit1 <= regDisplay[7:4];
    if(r_is_true)begin
        digit2 <= bit_index_rx;
        digit3 <= 16;
    end
    else begin
        digit2 <= bit_index_tx;
        digit3 <= 17;
    end
    if(r_is_true && (bit_index_rx == 0))begin
        regDisplay[7:0] <= rxMemory[7:0];
    end
    if(r_is_true && (bit_index_rx == 1))begin
        regDisplay[7:0] <= rxMemory[15:8];
    end
    if(r_is_true && (bit_index_rx == 2))begin
        regDisplay[7:0] <= rxMemory[23:16];
    end
    if(r_is_true && (bit_index_rx == 3))begin
        regDisplay[7:0] <= rxMemory[31:24];
    end
    if(!r_is_true && (bit_index_tx == 0))begin
        regDisplay[7:0] <= txMemory[7:0];
    end
    if(!r_is_true && (bit_index_tx == 1))begin
        regDisplay[7:0] <= txMemory[15:8];
    end
end

```

```

if(!r_is_true && (bit_index_tx == 2))begin
    regDisplay[7:0] <= txMemory[23:16];
end
if(!r_is_true && (bit_index_tx == 3))begin
    regDisplay[7:0] <= txMemory[31:24];
end
if(r_is_true && rightButtonOut)begin
    bit_index_rx <= bit_index_rx+1;
end
if(!r_is_true && rightButtonOut)begin
    bit_index_tx <= bit_index_tx+1;
end
if(r_is_true && leftButtonOut)begin
    bit_index_rx <= bit_index_rx-1;
end
if(!r_is_true && leftButtonOut)begin
    bit_index_tx <= bit_index_tx-1;
end
end
end

```

```

always @(posedge clock)
begin
    if (counter > 1)
        counter <= 2'b00;
    else
        counter <= counter + 1;
    refresh <= refresh + 1;
end

```

```

    case_index <= refresh[19:18];
end

always_ff @(posedge clock)
begin
    case (case_index)
        0: begin anodes = 4'b1110; led = digit0; end
        1: begin anodes = 4'b1101; led = digit1; end
        2: begin anodes = 4'b1011; led = digit2; end
        3: begin anodes = 4'b0111; led = digit3; end
    endcase
end

always_ff @(posedge clock)
begin
    case (led)
        0 : segments = 7'b0000001;
        1 : segments = 7'b1001111;
        2 : segments = 7'b0010010;
        3 : segments = 7'b0000110;
        4 : segments = 7'b1001100;
        5 : segments = 7'b0100100;
        6 : segments = 7'b0100000;
        7 : segments = 7'b0001111;
        8 : segments = 7'b0000000;
        9 : segments = 7'b0000100;
        10 : segments = 7'b0001000; // A
        11 : segments = 7'b1100000; // B
        12 : segments = 7'b1110010; // C
        13 : segments = 7'b1000010; // D
    endcase
end

```

```

    14 : segments = 7'b0110000; // E
    15 : segments = 7'b0111000; // F
    16 : segments = 7'b1111010; // r
    17 : segments = 7'b1110000; // t
    default : segments = 7'b1111111;
endcase
end
endmodule

```

TopModule

```

module top_module(
    input logic clock,
    input logic loadButton,
    input logic transmitButton,
    input logic rightButton,
    input logic leftButton,
    input logic inRDisplay,
    input logic [7:0] loaded_data,
    output logic [7:0] received_first,
    output logic [7:0] discarded_sent,
    output logic [3:0] anodes,
    output logic [6:0] seg
);
    wire connection;

    logic [31:0] rxMemory;
    logic [31:0] txMemory;

    logic LoadButtonOut;
    logic TransmitButtonOut;

```

```
logic RightButtonOut;
```

```
logic LeftButtonOut;
```

```
debounce d1(clock, loadButton, LoadButtonOut);
```

```
debounce d2(clock,transmitButton,TransmitButtonOut);
```

```
debounce d3(clock, leftButton, LeftButtonOut);
```

```
debounce d4(clock,rightButton,RightButtonOut);
```

```
debounce d5(clock,inRDisplay,inRDisplay_out);
```

```
ButtonUpLogic buttonUpLog(clock,inRDisplay_out,ButtonUpLogic);
```

```
displayMemory s1(clock, RightButtonOut , LeftButtonOut , txMemory, rxMemory, anodes,  
seg, ButtonUpLogic);
```

```
tx sender(clock, LoadButtonOut, TransmitButtonOut, loaded_data, connection,  
discarded_sent, txMemory);
```

```
rx receiver(clock, connection, received_first, rxMemory);
```

```
endmodule
```